
# SUBSTR8 v1.1 'Assíle'

*Developer Reference Guide*

*Developer: Costantino Pala, Geoscientist, PhD*

*costantino.pala.geo@proton.me*

# 1. What is SUBSTR8?

SUBSTR8 (version 1.1 'Assíle') is a modular Python framework for geospatial analysis built on top of rasterio, GeoPandas, Dask, NumPy, and a CustomTkinter graphical interface. It was developed in the context of a PhD project funded by CNR-IRPI-PG and DSCG-UNICA (University of Cagliari, Italy).

The name 'Assíle' is a Sardinian word indicating the European Pine Marten. I choose this mammal as it is a small, smart and fast animal.

The framework is designed to:

- Provide a clean, button-driven GUI for loading and processing raster and vector geospatial data.
- Centralise all I/O state in shared configuration dictionaries accessible by every module.
- Offer a library of reusable, parallelism-aware geospatial operations (array maths, slope, moving windows, raster-to-shapefile conversion, resampling, cropping, and map export).
- Allow researchers to plug in their own processing modules with minimal boilerplate.

SUBSTR8 is composed of six Python modules, each with a distinct responsibility:

| Module | Responsibility |
|---|---|
| `Launcher.py` | Entry point. Detects the OS and launches the GUI. |
| `assetios.py` | Shared configuration store. Holds all I/O paths and runtime parameters as module-level dictionaries. |
| `GUI.py` | Main graphical interface. Renders the button grid, toggle switches, and ties user actions to backend functions. |
| `file_manager.py` | Core function library. Handles file I/O, raster operations, memory optimisation, and the Preliminary Operations sub-window. |
| `about.py` | Help & About window. Displays developer info, quick-help text, bibliography, and the software license. |
| `mapadore.py` | Map export module. Reprojects rasters to EPSG:4326, applies colour scales, and saves publication-ready PNG maps. |

## 2. Architecture & Data Flow

The diagram below describes the high-level call flow when the application starts:

```
Launcher.py
  └─ detects OS → stores in assetios.parameters['os_system']
  └─ calls GUI.GUI()

GUI.py (main window)
  ├─ reads / writes  assetios.{parameters, input_tiff, input_shp, output_tiff}
  ├─ browse()         ──► file_manager.update_assetios()  (on file load)
  ├─ browse()         ──► file_manager.file_manager()     (Preliminary Ops
button)
  ├─ browse()         ──► about.about()                   (Help & About button)
  └─ buttfunction()   ──► your processing modules         (right-panel buttons)

file_manager.py
  ├─ open_array(), save(), update_assetios()
  ├─ array_calculator(), slope(), fentana(), shapadore()
  ├─ rastercutter(), resample()
  └─ file_manager()  [Preliminary Operations sub-window]

mapadore.py
  └─ mapper()  ──► convert_to_4326()  ──► saves PNG to <out_fold>/maps/
```

All modules communicate through the shared dictionaries in assetios.py — no direct return-value passing between the GUI layer and processing modules is required.

# 3. Module Reference

## 3.1 assetios.py — Shared Configuration Store

This module declares four module-level dictionaries that act as the application's shared state. Every other module imports from here. There are no functions to call; simply import the dictionary you need.

### parameters

```
from assetios import parameters

parameters = {
    "os_system"  : None,       # int  — 1=Linux, 2=Windows, 3=macOS (set by
Launcher)
    "cores"      : os.cpu_count(),  # int — CPU core count
    "resolution" : None,       # float — spatial resolution (metres)
    "epsg"       : None,       # str  — EPSG code, e.g. "32632"
    "out_fold"   : None,       # str  — absolute path to output directory
    "Setting_1"  : "Default",  # str  — toggle value ("Default" | "Custom")
    "Setting_2"  : "Mode_A",   # str  — toggle value ("Mode_A" | "Mode_B")
    "Toggle_1"   : "Off",      # str  — toggle value ("On" | "Off")
}
```

### input_tiff

```
from assetios import input_tiff

input_tiff = {
    "Input_Raster_1"  : None,   # str | None — path to primary input raster
    "Input_Raster_2"  : None,   # str | None
    "Input_Raster_3"  : None,   # str | None
    "Optional_Raster" : None,   # str | None — optional secondary raster
}
```

### input_shp

```
from assetios import input_shp

input_shp = {
    "Input_Vector_1" : None,   # str | None — path to primary vector layer
    "Input_Vector_2" : None,   # str | None
}
```

### output_tiff

```
from assetios import output_tiff

output_tiff = {
    "Output_Raster_1" : None,  # str | None — path to intermediate output 1
    "Output_Raster_2" : None,  # str | None — path to intermediate output 2
    "Output_Raster_3" : None,  # str | None
    "Final_Map"       : None,  # str | None — path to the final result
}
```

To add new keys, simply extend the dictionaries in assetios.py and add the corresponding buttons or logic in GUI.py.

## 3.2 Launcher.py — Entry Point

The sole purpose of this file is to detect the host operating system, store it in assetios.parameters, and start the main GUI. It also clears potentially conflicting PROJ environment variables before importing geospatial libraries.

### set_platform()

| | |
|---|---|
| **Signature** | `set_platform() -> None` |
| **Input** | `None — reads platform.system() automatically` |
| **Output** | `None — side-effect: sets parameters['os_system']` |
| **Side effects** | `parameters['os_system'] = 1 (Linux) \| 2 (Windows) \| 3 (macOS)` |

### start_app()

| | |
|---|---|
| **Signature** | `start_app() -> None` |
| **Input** | `None` |
| **Output** | `None — launches the GUI event loop (blocking)` |
| **Calls** | `set_platform(), GUI.GUI()` |

Usage: run Launcher.py directly to start SUBSTR8.

```
python Launcher.py
```

## 3.3 GUI.py — Main Graphical Interface

Builds and runs the main application window using CustomTkinter. It renders two button grids (left panel for I/O, right panel for processing), a top bar with toggle switches, and wires all user interactions to the appropriate backend calls. GUI.py does not contain any geospatial processing logic.

### GUI()

| | |
|---|---|
| **Signature** | `GUI() -> None` |
| **Input** | `None — reads assetios dicts at startup to colour buttons` |
| **Output** | `None — blocking Tk event loop` |
| **Side effects** | `Updates assetios.parameters, input_tiff, input_shp, output_tiff on user interactions` |

### Internal functions

The following functions are defined inside GUI() and are not callable from outside:

#### browse(btn_instance, label, ext, f_type, dark_color, dict_key)

Handles all button click events on the left panel. Routes to filedialog.askopenfilename, filedialog.askdirectory, file_manager.file_manager(), or about.about() depending on the button configuration.

| | |
|---|---|
| **btn_instance** | `ctk.CTkButton — the button widget that was clicked` |
| **label** | `str — display text of the button` |
| **ext** | `str — file extension (e.g. ".tif") or "function"` |
| **f_type** | `str — file type description or the function name` |
| **dark_color** | `str — hex colour applied when a file is loaded` |
| **dict_key** | `str \| None — key in the assetios dictionary to update` |

#### buttfunction(label)

Called when any right-panel processing button is clicked. Acts as a dispatcher: match the button label to the appropriate processing module call. By default the body contains pass placeholders; replace each with your module.run() call.

| | |
|---|---|
| **label** | `str — text of the clicked button` |
| **Returns** | `None` |

#### annoadore(window) / annoa()

A polling loop (1-second interval) that monitors the keys listed in TARGET_KEYS. If the corresponding path in assetios is not None it darkens the button colour, giving the user visual feedback that an intermediate output is available.

#### create_smart_button(index, button_data)

Factory function that creates a single button in the left-panel grid. Reads the current state of assetios to decide whether to render the button in its 'loaded' or 'empty' colour.

Customisation points in GUI.py:

- left_buttons_text — add rows to extend the left panel. Each row: (label, ext, f_type, theme_key, dict_key).
- right_buttons_text — add rows to extend the right panel. Each row: (label, hex_colour).
- switcher_config — add rows to add toggle switches. Each row: (title, sx_label, dx_label, true_value, false_value).
- color_themes — maps a theme key to (empty_colour, loaded_colour) tuples.
- TARGET_KEYS — list of assetios keys whose buttons should be auto-monitored.

## 3.4 file_manager.py — Core Function Library

This is the largest and most reusable module. It provides functions for file I/O, raster arithmetic, spatial analysis, memory management, and the Preliminary Operations GUI. All processing modules in your project should import from here.

### I/O Functions

#### resource_path(relative_path)

| | |
|---|---|
| **Signature** | `resource_path(relative_path: str) -> str` |
| **Input** | `relative_path` — path relative to the script (or PyInstaller bundle) |
| **Output** | `str` — absolute path, compatible with both dev and frozen (.exe) environments |

#### load_file(entry)

| | |
|---|---|
| **Signature** | `load_file(entry: ctk.CTkEntry) -> np.ndarray \| None` |
| **Input** | `entry` — a CustomTkinter Entry widget; the selected path is inserted into it |
| **Output** | `np.ndarray` — band 1 of the selected raster, or None if cancelled / error |
| **Side effects** | Opens a file dialog; populates the Entry widget with the chosen path; prints shape and transform to console |

#### load_directory(entry)

| | |
|---|---|
| **Signature** | `load_directory(entry: ctk.CTkEntry) -> None` |
| **Input** | `entry` — CTkEntry widget to populate |
| **Output** | None — side-effect: inserts selected directory path into entry |

#### open_array(file)

| | |
|---|---|
| **Signature** | `open_array(file: str) -> tuple[rasterio.DatasetReader, np.ndarray, Affine]` |
| **Input** | `file` — absolute path to a raster file (.tif, .jp2, .asc, …) |
| **Output** | (src, array, transform) — open rasterio dataset, band-1 float64 array, affine transform; or (None, None) on error |

```
src, arr, trs = open_array('path/to/dem.tif')
# arr is a float64 numpy array of band 1
```

### save(result, otn, resolution, epsg, otrs)

| | |
|---|---|
| **Signature** | `save(result: np.ndarray \| da.Array, otn: str, resolution: float, epsg: int, otrs: Affine) -> None` |
| **result** | NumPy or Dask array to persist |
| **otn** | str — output filename without extension (e.g. 'slope_output') |
| **resolution** | float — pixel size in CRS units |
| **epsg** | int — EPSG code of the target CRS |
| **otrs** | Affine — geospatial transform |
| **Output** | None — writes "<otn>.tif" to parameters["out_fold"]; nodata = -9999; dtype = float32; embeds mean/min/max/std as metadata |

```
save(result_array, 'slope', 10, 32632, src.transform)
# → <out_fold>/slope.tif
```

### update_assetios(output_name, file_path, file_type, io_type)

| | |
|---|---|
| **Signature** | `update_assetios(output_name: str, file_path: str, file_type: str, io_type: str) -> bool` |
| **output_name** | str — dictionary key (e.g. "Input_Raster_1") |
| **file_path** | str — absolute path to the file |
| **file_type** | "tiff" \| "JPEG 2000" \| "shp" |
| **io_type** | "input" \| "output" |
| **Returns** | bool — True on success, False if key or type unknown |

```
update_assetios("Input_Raster_1", "/data/dem.tif", "tiff", "input")
# → assetios.input_tiff['Input_Raster_1'] = '/data/dem.tif'
```

## Spatial Analysis Functions

### shapadore(raster_path, shapefile_path, epsg)

| | |
|---|---|
| **Signature** | `shapadore(raster_path: str, shapefile_path: str, epsg: int) -> None` |
| **raster_path** | str — path to binary raster (pixel value = 1 for features to vectorise) |
| **shapefile_path** | str — full output path including filename and .shp extension |
| **epsg** | int — EPSG code for the output shapefile |
| **Output** | None — writes an ESRI Shapefile; only pixels equal to 1 are polygonised (nodata excluded) |

```
shapadore("classified.tif", "/out/features.shp", 32632)
```

### array_calculator(iarray1, iarray2, operation, resolution, epsg, otrs, otn='None')

| | |
|---|---|
| **Signature** | `array_calculator(iarray1, iarray2, operation, resolution, epsg, otrs, otn='None')` |
| **iarray1/2** | `np.ndarray \| da.Array \| int \| float — operands (must have the same shape if both are arrays)` |
| **operation** | `"sum" \| "difference" \| "times" \| "division"` |
| **resolution** | `float — pixel size` |
| **epsg** | `int — EPSG code` |
| **otrs** | `Affine — geospatial transform` |
| **otn** | `str — output name; omit (default 'None') to skip saving` |
| **Returns (otn set)** | `(str, np.ndarray) — (output .tif path, result array)` |
| **Returns (otn='None')** | `np.ndarray — result array only` |

Uses Dask chunked parallelism. Chunk size is automatically derived from available RAM via calculate_chunk_size().

```
# Element-wise difference, save result
path, arr = array_calculator(arr_a, arr_b, 'difference', 10, 32632, trs,
'diff_output')

# In-memory division (no save)
result = array_calculator(arr_a, 2.0, 'division', 10, 32632, trs)
```

### slope(demar)

| | |
|---|---|
| **Signature** | `slope(demar: np.ndarray) -> da.Array` |
| **demar** | `np.ndarray — Digital Elevation Model array` |
| **Returns** | `da.Array — slope in degrees; Sobel gradient + 3x3 mean smoothing kernel applied before conversion` |

```
_, dem_arr, trs = open_array('dem.tif')
slope_arr = slope(dem_arr)   # returns a Dask array
save(slope_arr.compute(), 'slope', 10, 32632, trs)
```

### fentana(arr, funtz, epsg=None, otrs=None, resolution=None, window_size=5)

| | |
|---|---|
| **Signature** | `fentana(arr, funtz, epsg=None, otrs=None, resolution=None, window_size=5)` |
| **arr** | `np.ndarray \| da.Array — input raster` |
| **funtz** | `da.mean \| da.std — statistic to compute in each window` |
| **window_size** | `int — side length of the square moving window (default 5)` |
| **Returns** | `da.Array — local mean or local standard deviation raster; reflect padding at edges` |

'Fentana' means 'window' in Sardinian.

```
import dask.array as da
local_mean = fentana(arr, da.mean, window_size=7)
local_std  = fentana(arr, da.std,  window_size=7)
```

## Memory Management

### calculate_chunk_size(dtype=np.float32, target_memory_fraction=0.5, max_chunk_size=1000)

| | |
|---|---|
| **Signature** | `calculate_chunk_size(dtype=np.float32, target_memory_fraction=0.5, max_chunk_size=1000) -> tuple[int,int]` |
| **dtype** | `np.dtype — array element type (default float32)` |
| **target_memory_fraction** | `float — fraction of total RAM to target for a single chunk (default 0.5)` |
| **max_chunk_size** | `int — hard cap on chunk width/height in pixels (default 1000)` |
| **Returns** | `tuple[int, int] — (chunk_width, chunk_height) for use with da.from_array(..., chunks=...)` |

Called automatically by array_calculator(), slope(), and fentana(). Expose it directly if you need custom Dask chunking in your own modules.

## Preprocessing Utilities

### rastercutter(raster1_entry, raster2_entry, output_dir)

| | |
|---|---|
| **Signature** | `rastercutter(raster1_entry: str, raster2_entry: str, output_dir: str) -> None` |
| **raster1_entry** | `str — path to first raster` |
| **raster2_entry** | `str — path to second raster` |
| **output_dir** | `str — folder where the cropped raster will be saved` |
| **Output** | `None — the larger raster is cropped to the smaller extent and saved as "crop_<original_name>.tif"; a messagebox confirms success or reports errors` |

```
rastercutter("/data/large_dem.tif", "/data/small_ref.tif", "/out/")
# → /out/crop_large_dem.tif
```

### resample(raster_path, resolution, output_dir)

| | |
|---|---|
| **Signature** | `resample(raster_path: str, resolution: int\|str, output_dir: str) -> None` |
| **raster_path** | `str — path to the raster to resample` |
| **resolution** | `int \| str — target pixel size in CRS units (converted to int internally)` |
| **output_dir** | `str — destination folder` |
| **Algorithm** | `Bilinear resampling (rasterio.enums.Resampling.bilinear)` |
| **Output** | `None — saved as "resampled_<name>_<resolution>m_res.tif"; messagebox confirmation` |

```
resample("/data/dem.tif", 10, "/out/")
# → /out/resampled_dem_10m_res.tif
```

## GUI Utility Functions (used internally by file_manager())

### set_resolution(resolution_entry)

| | |
|---|---|
| **Signature** | `set_resolution(resolution_entry: ctk.CTkEntry) -> float \| None` |
| **Input** | `CTkEntry widget containing the desired resolution as a string` |
| **Output** | `float — the validated resolution value; also stored in parameters['resolution']; None on invalid input` |

### set_epsg(epsg_entry)

| | |
|---|---|
| **Signature** | `set_epsg(epsg_entry: ctk.CTkEntry) -> str` |
| **Input** | `CTkEntry widget containing the EPSG code as a string` |
| **Output** | `str — EPSG code stored in parameters['epsg']` |

### set_thr(thr_entry)

| | |
|---|---|
| **Signature** | `set_thr(thr_entry: ctk.CTkEntry) -> float` |
| **Input** | `CTkEntry widget containing a threshold value` |
| **Output** | `float — threshold stored in parameters['ndvi_thr']` |
| **Note** | `Originally designed for NDVI thresholding; repurpose the key name in assetios if needed` |

### file_manager()

| | |
|---|---|
| **Signature** | `file_manager() -> None` |
| **Input** | `None` |
| **Output** | `None — opens the 'Preliminary Operations' sub-window (blocking Tk loop)` |
| **Features** | `Set EPSG, Set Resolution, Set NDVI Threshold, Cropping Tool (rastercutter), Resampling Tool (resample)` |

## 3.5 about.py — Help & About Window

Renders a 2×2 panel Toplevel window with four informational sections: developer info / logo (top-left), quick-help guide (top-right), bibliography (bottom-left), and the license text (bottom-right).

### resource_path(relative_path)

| | |
|---|---|
| **Signature** | `resource_path(relative_path: str) -> str` |
| **Input** | `Relative path to an asset (icon, logo PNG)` |
| **Output** | `str — absolute path, compatible with PyInstaller frozen bundles` |

### about()

| | |
|---|---|
| **Signature** | `about() -> None` |
| **Input** | `None — reads INFO_TEXT, QUICK_HELP_TEXT, BIBLIOGRAPHY_TEXT, LICENSE_TEXT constants` |
| **Output** | `None — opens a 1000×800 px Toplevel window; non-blocking (the main GUI remains usable)` |
| **Customise** | `Edit the four TEXT constants at the top of about.py to adapt to your project` |

The four text constants to customise:

- INFO_TEXT — software name, developer details, institution, contact.
- QUICK_HELP_TEXT — step-by-step user instructions.
- BIBLIOGRAPHY_TEXT — scientific references used in your modules.
- LICENSE_TEXT — full license text (defaults to AGPL v3 placeholder).

## 3.6 mapadore.py — Map Export Module

Handles cartographic output. It optionally fetches Open Sans fonts from GitHub at runtime, reprojects rasters to EPSG:4326 for display, and renders publication-quality PNG maps with Matplotlib.

'Mapadore' means 'cartographer' in Sardinian.

### save_font_temp(url)

| | |
|---|---|
| **Signature** | `save_font_temp(url: str) -> str` |
| **Input** | `url — direct download URL to a .ttf font file` |
| **Output** | `str — local path to the downloaded font ("temp_font.ttf" in cwd)` |
| **Note** | `Called automatically at module import time; falls back to the system default font on network failure` |

### convert_to_4326(input_tif, output_tif)

| | |
|---|---|
| **Signature** | convert_to_4326(input_tif: str, output_tif: str) -> None |
| **input_tif** | str — path to the source raster in any CRS |
| **output_tif** | str — path for the reprojected output raster (EPSG:4326, nearest-neighbour) |
| **Output** | None — writes a GeoTIFF in WGS-84 geographic coordinates |

### mapper(tif_path, title, color_gradient, scale_type='linear', font_props=None, value_to_label=None)

| | |
|---|---|
| **Signature** | mapper(tif_path, title, color_gradient, scale_type='linear', font_props=None, value_to_label=None) |
| **tif_path** | str — path to input raster (any CRS; will be reprojected internally) |
| **title** | str — map title (also used as the output PNG filename) |
| **color_gradient** | str — Matplotlib colormap name (e.g. 'viridis', 'RdYlGn', 'terrain') |
| **scale_type** | "linear" — min-max normalisation \| "quantile" — 0/25/50/75/100 percentile breaks |
| **font_props** | dict with keys "bold_italic", "italic", "bold_italic_legend", "normal" → FontProperties objects |
| **value_to_label** | dict \| None — {pixel_value: 'Label'} for classified/categorical maps; triggers a legend instead of a colorbar |
| **Output** | None — saves "<title>.png" at 300 DPI to <parameters["out_fold"]>/maps/ |

```
from mapadore import mapper
from matplotlib.font_manager import FontProperties

font_props = {
    'bold_italic':        FontProperties(family='Open Sans', style='italic',
weight='bold', size=15),
    'italic':             FontProperties(family='Open Sans', style='italic',
size=12),
    'bold_italic_legend': FontProperties(family='Open Sans', style='italic',
weight='bold', size=13),
    'normal':             FontProperties(family='Open Sans', size=12),
}

# Continuous raster (linear scale)
mapper('slope.tif', 'Slope Map', 'terrain', scale_type='linear',
font_props=font_props)

# Classified raster (categorical legend)
labels = {1: 'Low', 2: 'Medium', 3: 'High'}
mapper('classified.tif', 'Risk Class', 'RdYlGn', value_to_label=labels,
font_props=font_props)
```

# 4. Extending SUBSTR8

## 4.1 Adding a New Processing Module

Follow these four steps to add a new analytical module:

1. Create your module file (e.g. `module_ndvi.py`) in the project folder.
2. Import assetios and file_manager at the top of your module.
3. Add a row to `right_buttons_text` in GUI.py with your button label and colour.
4. Add a branch to `buttfunction()` in GUI.py that calls your module.

```
# module_ndvi.py
import file_manager as fm
import assetios

def run():
    red_path = assetios.input_tiff['Input_Raster_1']
    nir_path = assetios.input_tiff['Input_Raster_2']
    epsg     = int(assetios.parameters['epsg'])
    res      = assetios.parameters['resolution']

    _, red, trs = fm.open_array(red_path)
    _, nir, _   = fm.open_array(nir_path)

    ndvi = fm.array_calculator(nir - red, nir + red, 'division', res, epsg, trs,
'ndvi')
    assetios.output_tiff['Output_Raster_1'] = ndvi[0]
```

```
# In GUI.py → right_buttons_text
    ("Execute\nNDVI", "#512DA8"),

# In GUI.py → buttfunction()
    elif label == "Execute\nNDVI":
        module_ndvi.run()
```

## 4.2 Adding New assetios Keys

- Open assetios.py and add the key with a None (or default) value to the relevant dictionary.
- Add a corresponding button row in left_buttons_text (GUI.py) with the same key as dict_key.
- If you want the button to be auto-monitored (colour change on file presence), add the key to TARGET_KEYS in GUI.py.

## 4.3 Packaging as an Executable

SUBSTR8 is designed to be packaged with PyInstaller. The resource_path() helper in each module ensures assets (icons, logos) are found in both development and frozen environments.

```
pyinstaller --onefile --windowed \
  --add-data 'SUBSTR8_logo.ico;.' \
  --add-data 'SUBSTRAT8_logo.png;.' \
  Launcher.py
```

## 5. Dependencies

| Package | Version (tested) | Used in |
|---|---|---|
| `customtkinter` | ≥ 5.x | GUI.py, file_manager.py, about.py |
| `rasterio` | ≥ 1.3 | file_manager.py, mapadore.py |
| `geopandas` | ≥ 0.14 | file_manager.py, mapadore.py |
| `numpy` | ≥ 1.24 | file_manager.py, mapadore.py |
| `dask` | ≥ 2024 | file_manager.py |
| `dask-image` | ≥ 2023 | file_manager.py |
| `scipy` | ≥ 1.11 | file_manager.py |
| `psutil` | ≥ 5.x | file_manager.py |
| `matplotlib` | ≥ 3.7 | mapadore.py |
| `Pillow (PIL)` | ≥ 10.x | GUI.py, about.py |
| `requests` | ≥ 2.x | mapadore.py (font download) |

```
pip install customtkinter rasterio geopandas numpy dask dask-image scipy psutil
matplotlib Pillow requests
```

## 6. License and Contacts


Developed in the framework of a PhD project funded by CNR-IRPI-PG and DSCG-UNICA.
Coding support and suggestions: ChatGPT, Gemini.
Guide provided to you by Claude (Anthropic)

In case you notice bugs or have some suggestions please feel free to text me:
costantino.pala.geo@proton.me

https://www.github.com/constantineshovel